

LAPORAN FINAL PROJECT

IMPLEMENTASI ARSITEKTUR READ-WRITE SPLITTING

POSTGRESQL MENGGUNAKAN PGPOOL-II PADA

APLIKASI KOMPOSIN (SISTEM BANK SAMPAH &

E-COMMERCE PENGELOLAAN SAMPAH)



Disusun Oleh:

Fabian Nabil 232410102022

[Ivan] [NIM 2]

[Rio] [NIM 3]

Dosen Pembimbing:

[Nama Dosen Pembimbing]

NIP. [NIP Dosen Pembimbing]

PROGRAM STUDI TEKNOLOGI INFORMASI

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS JEMBER

2026

ABSTRAK

Final project ini mengimplementasikan arsitektur *read-write splitting* pada PostgreSQL menggunakan Pgpool-II sebagai *middleware* untuk aplikasi Komposin — sebuah platform bank sampah dan *e-commerce* pengelolaan sampah berbasis Laravel 10. Arsitektur terdiri dari 1 (satu) PostgreSQL Master, 2 (dua) PostgreSQL Replica, dan 1 (satu) Pgpool-II yang di-*deploy* menggunakan Docker Compose pada jaringan *bridge* internal. Metode *streaming replication* asinkron digunakan untuk menjaga sinkronisasi data dari Master ke Replica. Pgpool-II dikonfigurasi dengan `master_slave_mode` dan `load_balance_mode` untuk memisahkan query *write* (INSERT, UPDATE, DELETE) ke Master dan query *read* (SELECT) ke Replica secara transparan. Hasil pengujian menunjukkan bahwa mekanisme *read-write splitting* berhasil memisahkan beban query, *load balancing* mendistribusikan query SELECT secara merata ke kedua Replica, dan sistem tetap beroperasi ketika salah satu Replica dimatikan karena mekanisme *health check* dan *failover* Pgpool-II.

Kata Kunci: *read-write splitting*, PostgreSQL, Pgpool-II, *streaming replication*, *load balancing*, *fault tolerance*, Docker Compose, Laravel

Daftar Isi

ABSTRAK	i
DAFTAR ISI	iii
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Ruang Lingkup Sistem	2
1.2.1 Aplikasi yang Dibangun	2
1.2.2 Fitur Utama Aplikasi	2
1.2.3 Batasan Implementasi Sistem	3
2 ARSITEKTUR DAN TOPOLOGI SISTEM	4
2.1 Diagram Topologi Jaringan	4
2.1.1 Alamat IP dan Port Layanan	4
2.1.2 Arah Komunikasi Antar Server	5
2.2 Spesifikasi Lingkungan (Environment)	6
2.2.1 Struktur File Konfigurasi	7
3 IMPLEMENTASI DAN KONFIGURASI	8
3.1 Konfigurasi Replication	8
3.1.1 Konfigurasi PostgreSQL Master	8
3.1.2 Konfigurasi PostgreSQL Replica	11
3.2 Konfigurasi Pgpool-II	13
3.2.1 Konfigurasi Backend	13
3.2.2 Konfigurasi Load Balancing dan Read-Write Splitting	14
3.2.3 Konfigurasi Autentikasi Client (pool_hba.conf)	15
3.3 Integrasi Aplikasi	16
3.3.1 Konfigurasi Environment	16
3.3.2 Konfigurasi Datasource Laravel	16
3.3.3 Koneksi di docker-compose.yml	17

4	PENGUJIAN DAN ANALISIS	19
4.1	Pengujian Read-Write Splitting (Skenario A)	19
4.1.1	Pengujian Query Write	19
4.1.2	Pengujian Query Read	22
4.1.3	Kesimpulan Pengujian Skenario A	24
4.2	Pengujian Toleransi Kesalahan / Replica Down (Skenario B)	24
4.2.1	Kronologi Pengujian	24
4.2.2	Kondisi Sistem Sebelum Replica Dimatikan	25
4.2.3	Kondisi Sistem Setelah Replica Dimatikan	25
4.2.4	Bukti Aplikasi Tetap Berjalan	26
4.2.5	Bukti Pengalihan Traffic ke Replica Lain	26
4.2.6	Kondisi Setelah Replica 1 Dinyalakan Kembali	27
4.2.7	Kesimpulan Pengujian Skenario B	27
5	KESIMPULAN DAN KENDALA	29
5.1	Kesimpulan	29
5.1.1	Keberhasilan Implementasi Replikasi Database	29
5.1.2	Keberhasilan Mekanisme Read-Write Splitting	29
5.1.3	Manfaat Penggunaan Pgpool-II	30
5.1.4	Peningkatan Efisiensi Distribusi Query Database	30
5.2	Kendala dan Solusi	30
5.2.1	Kendala yang Mungkin Ditemukan	30
5.2.2	Proses Analisis dan Penyelesaian Masalah	32

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat mendorong kebutuhan akan sistem yang mampu menangani volume data dan lalu lintas query yang tinggi secara efisien. Pada aplikasi *e-commerce* dan sistem pengelolaan data seperti Komposin — sebuah platform bank sampah dan *e-commerce* pengelolaan sampah — beban operasi *read* (pembacaan data) umumnya jauh lebih tinggi dibandingkan operasi *write* (penulisan data). Pengguna lebih sering melakukan pencarian produk, melihat katalog, membaca artikel, dan mengecek riwayat transaksi dibandingkan melakukan pembelian atau pendaftaran akun. Karakteristik beban kerja seperti ini lazim ditemukan pada hampir semua aplikasi web modern, di mana rasio *read* terhadap *write* dapat mencapai 80:20 atau bahkan 90:10. Oleh karena itu, strategi pengelolaan database perlu disesuaikan agar sumber daya komputasi dialokasikan secara proporsional terhadap jenis beban yang dominan.

Apabila hanya menggunakan satu *server* database (PostgreSQL *single instance*), seluruh beban query — baik *read* maupun *write* — diproses oleh *server* yang sama. Kondisi ini menimbulkan beberapa permasalahan, antara lain:

1. **Bottleneck performa.** Ketika jumlah pengguna meningkat, *server* tunggal tidak mampu menangani lonjakan query secara bersamaan, menyebabkan waktu respons yang lambat. Pengguna akan merasakan halaman yang lama dimuat, terutama pada jam-jam sibuk ketika banyak pengguna mengakses katalog dan melakukan transaksi secara bersamaan. Kondisi ini dapat menurunkan kepuasan pengguna dan berpotensi mengurangi pendapatan aplikasi.
2. **Tidak ada toleransi kesalahan (*fault tolerance*).** Jika *server* database mengalami kegagalan (*down*), seluruh aplikasi berhenti beroperasi karena tidak ada cadangan. Downtime yang terjadi dapat berlangsung dalam hitungan menit hingga jam, tergantung pada kecepatan pemulihan server.
3. **Beban *read* mengganggu operasi *write*.** Query *SELECT* yang berat (misalnya laporan transaksi) dapat memperlambat query *INSERT/UPDATE* yang membutuhkan komitmen data segera. Akibatnya, proses pembelian atau pendaftaran pengguna baru dapat tertunda oleh query pembacaan yang seharusnya tidak mendesak.

Untuk mengatasi permasalahan tersebut, diterapkan arsitektur *read-write splitting* dengan

replikasi database. Mekanisme ini memisahkan query *write* (INSERT, UPDATE, DELETE) yang dikirim ke **PostgreSQL Master**, dan query *read* (SELECT) yang didistribusikan ke satu atau lebih **PostgreSQL Replica**. Sebagai *middleware* yang mengatur distribusi query, digunakan **Pgpool-II** — sebuah *connection pooler* dan *load balancer* yang secara *transparan* mengarahkan query ke *backend* yang tepat berdasarkan jenis operasinya. Pgpool-II juga menyediakan mekanisme *health check* dan *failover* otomatis, sehingga jika salah satu Replica gagal, traffic akan dialihkan ke Replica lain tanpa mengganggu aplikasi. Keseluruhan arsitektur di-*deploy* menggunakan Docker Compose untuk memastikan konsistensi lingkungan dan kemudahan replikasi konfigurasi.

Dengan arsitektur ini, aplikasi Komposin diharapkan mampu menangani beban *read* yang tinggi tanpa mengorbankan performa *write*. Selain itu, sistem diharapkan tetap beroperasi meskipun salah satu *replica* mengalami kegagalan (*high availability*), karena Pgpool-II akan secara otomatis mengalihkan koneksi ke node yang masih sehat. Arsitektur ini juga memungkinkan distribusi beban query secara merata ke beberapa *node* database melalui mekanisme *load balancing*. Keempat, penggunaan *connection pooling* mengurangi overhead pembukaan koneksi baru yang berulang. Kelima, pemisahan beban *read* dan *write* memungkinkan masing-masing server dioptimalkan secara independen sesuai perannya.

1.2 Ruang Lingkup Sistem

1.2.1 Aplikasi yang Dibangun

Aplikasi Komposin adalah platform berbasis web yang dikembangkan menggunakan **Laravel 10** (PHP) dengan database **PostgreSQL 16**. Aplikasi ini memiliki dua peran pengguna: **Admin** dan **Pelanggan**, di mana Admin mengelola data produk, artikel, reward, transaksi, rute pengambilan sampah, serta laporan. Sementara itu, Pelanggan dapat melihat katalog produk, melakukan pembelian, menukarkan reward, melihat riwayat transaksi, dan berlangganan layanan pengambilan sampah. Seluruh fitur tersebut mengandalkan operasi database yang intensif, baik *read* maupun *write*, sehingga menjadi kandidat ideal untuk penerapan *read-write splitting*. Aplikasi dikemas dalam container Docker yang berjalan di atas Nginx dan Supervisor untuk memastikan keandalan layanan.

1.2.2 Fitur Utama Aplikasi

- **Manajemen Katalog Produk.** Admin dapat menambah, mengubah, dan menghapus produk. Pelanggan dapat melihat daftar produk. Fitur ini didominasi oleh operasi SELECT yang tinggi, terutama saat banyak pengguna menjelajahi katalog secara bersamaan.

- **Manajemen Transaksi.** Pelanggan melakukan pembelian produk melalui keranjang belanja dan *check-out*. Admin mengelola status transaksi. Setiap transaksi melibatkan kombinasi query `INSERT` (membuat pesanan) dan `SELECT` (memvalidasi stok dan data pelanggan).
- **Manajemen Reward.** Pelanggan menukarkan poin dengan reward, dan Admin mengonfirmasi penukaran reward. Proses ini memerlukan validasi saldo poin melalui `SELECT` dan pencatatan penukaran melalui `INSERT`.
- **Manajemen Artikel.** Admin membuat dan mengelola artikel edukasi seputar pengelolaan sampah. Pelanggan membaca artikel yang mayoritas hanya memerlukan operasi `SELECT`.
- **Berlangganan.** Pelanggan dapat berlangganan layanan pengambilan sampah berkala. Sistem mencatat jadwal dan status berlangganan melalui operasi `INSERT` dan `UPDATE`.
- **Pelaporan.** Admin melihat laporan transaksi dan penghasilan dari layanan berlangganan. Laporan melibatkan query `SELECT` agregat yang kompleks dan berpotensi membebani database.
- **Pengambilan Sampah.** Pelanggan menjadwalkan pengambilan sampah dan mendapatkan poin sebagai imbalan. Fitur ini melibatkan pencatatan jadwal (`INSERT`) dan perubahan poin (`UPDATE`).

1.2.3 Batasan Implementasi Sistem

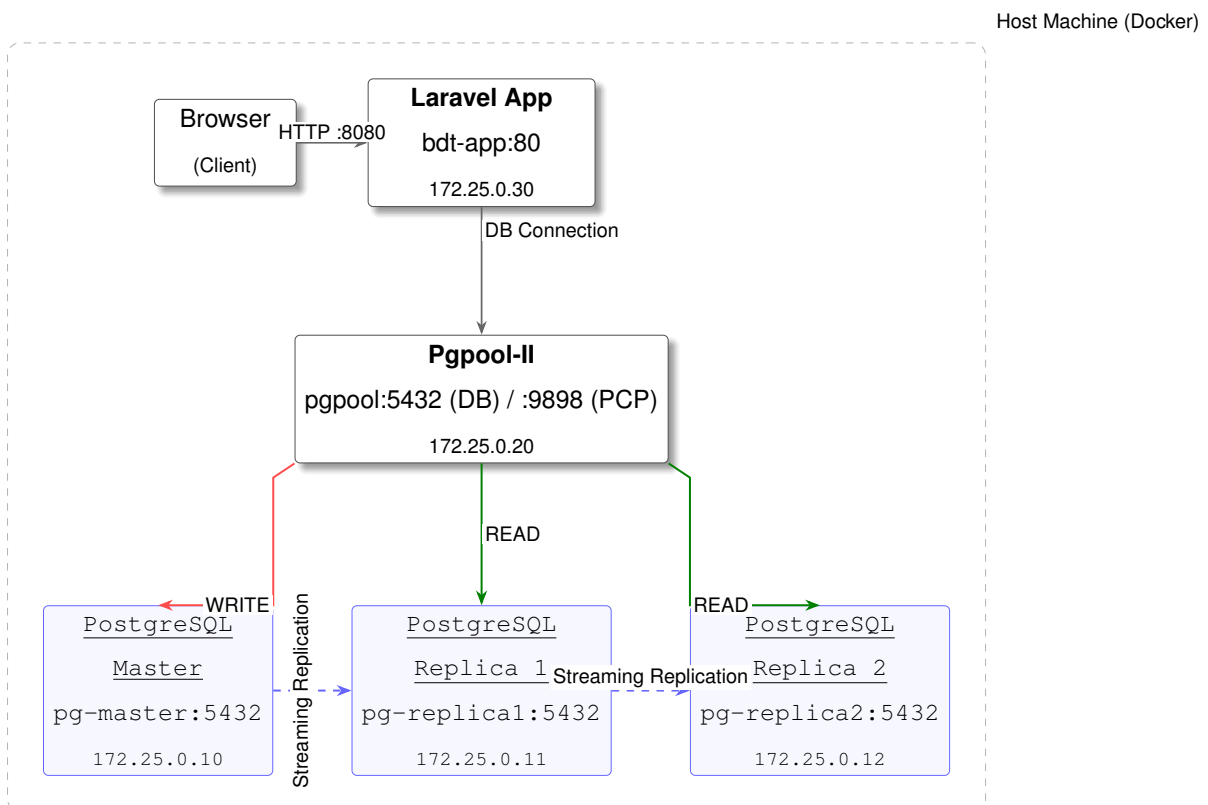
Batasan implementasi pada final project ini ditetapkan untuk menjaga fokus dan kelayakan teknis dalam lingkup mata kuliah Basis Data Terdistribusi. Sistem menggunakan **1 (satu) PostgreSQL Master** sebagai *node* utama yang menerima seluruh operasi *write*, serta **2 (dua) PostgreSQL Replica** sebagai *node* pembaca yang menerima operasi *read*. Satu **Pgpool-II** berperan sebagai *middleware* pengatur *connection pooling*, *load balancing*, dan *read-write splitting*. Seluruh layanan di-*deploy* menggunakan **Docker Compose** dalam satu *host* dengan jaringan *bridge* internal, sehingga konfigurasi dapat direproduksi dengan mudah di lingkungan lain. Replikasi menggunakan metode *streaming replication* secara **asinkron** yang sudah tersedia secara *native* pada PostgreSQL 16. Aplikasi **tidak** terhubung langsung ke PostgreSQL Master maupun Replica — seluruh koneksi database diarahkan ke Pgpool-II sebagai *single entry point*. Pendekatan ini memastikan bahwa mekanisme *read-write splitting* berjalan secara transparan tanpa modifikasi kode aplikasi.

BAB 2

ARSITEKTUR DAN TOPOLOGI SISTEM

2.1 Diagram Topologi Jaringan

Arsitektur sistem terdiri dari 5 (lima) *node* utama yang berjalan di atas jaringan Docker *bridge* dengan subnet $172.25.0.0/24$. Diagram pada Gambar 2.1 menggambarkan hubungan antar *node* beserta alamat IP, port layanan, dan arah komunikasi data yang telah dirancang. Setiap *node* memiliki peran spesifik — mulai dari *client* browser, aplikasi Laravel, middleware Pgpool-II, hingga tiga *backend* PostgreSQL (satu Master dan dua Replica). Jaringan Docker *bridge* dipilih karena menyediakan isolasi jaringan internal yang aman dengan resolusi nama *host* otomatis antar *container*. Konfigurasi IP statis diterapkan untuk memastikan setiap *node* selalu dapat dijangkau pada alamat yang konsisten meskipun terjadi restart *container*.



Gambar 2.1. Diagram Topologi Jaringan Sistem *Read-Write Splitting*

2.1.1 Alamat IP dan Port Layanan

Tabel 2.1 merinci alamat IP dan port setiap *node* dalam arsitektur. Setiap *container* memiliki dua port: port internal yang digunakan untuk komunikasi antar *container* dalam jaringan Docker, dan port *host* yang dipetakan untuk akses dari luar (digunakan selama pengembangan

dan pengujian). Aplikasi Laravel diekspos pada port host 8080 yang dipetakan ke port 80 internal Nginx di dalam *container*. Pgpool-II membuka dua port: 5432 untuk koneksi database PostgreSQL dan 9898 untuk antarmuka administrasi PCP (*Pgpool Control Protocol*). Masing-masing PostgreSQL *backend* diekspos pada port host yang berbeda (5433, 5434, 5435) untuk memungkinkan koneksi langsung saat debugging, meskipun dalam operasi normal aplikasi hanya terhubung melalui Pgpool-II.

Tabel 2.1. Spesifikasi Jaringan Tiap Node

Komponen	Container	IP Address	Port (Host)	Port (Container)
Laravel App	bdt-app	172.25.0.30	8080	80
Pgpool-II	pgpool	172.25.0.20	5436, 9898	5432, 9898
PostgreSQL Master	pg-master	172.25.0.10	5433	5432
PostgreSQL Replica 1	pg-replica1	172.25.0.11	5434	5432
PostgreSQL Replica 2	pg-replica2	172.25.0.12	5435	5432

2.1.2 Arah Komunikasi Antar Server

Komunikasi antar *node* mengikuti alur yang telah dirancang untuk memastikan pemisahan beban kerja dan keamanan data. Berikut adalah rincian arah komunikasi pada setiap jalur:

1. **Client → Aplikasi (HTTP):** Browser mengakses aplikasi Laravel melalui port 8080 pada *host*. Permintaan HTTP ini kemudian diteruskan oleh Nginx ke aplikasi Laravel melalui PHP-FPM yang dikelola Supervisor.
2. **Aplikasi → Pgpool-II (PostgreSQL Wire Protocol):** Seluruh koneksi database dari aplikasi diarahkan ke Pgpool-II (172.25.0.20:5432), bukan langsung ke PostgreSQL Master. Aplikasi tidak perlu mengetahui topology database di belakang Pgpool-II karena middleware ini menangani seluruh logika routing query secara transparan.
3. **Pgpool-II → PostgreSQL Master (WRITE):** Query INSERT, UPDATE, DELETE diteruskan ke Master (172.25.0.10:5432). Pgpool-II mengenali jenis operasi SQL dan mengarahkannya ke *backend* 0 (Master) yang memiliki flag ALLOW_TO_FAILOVER.
4. **Pgpool-II → PostgreSQL Replica (READ):** Query SELECT didistribusikan ke Replica 1 (172.25.0.11:5432) dan Replica 2 (172.25.0.12:5432) berdasarkan *weight* masing-masing. Dengan bobot yang sama (*weight*=1), kedua Replica mendapat porsi query yang seimbang.

5. **Master → Replica (Streaming Replication):** Data WAL (*Write-Ahead Log*) dikirim dari Master ke kedua Replica secara *streaming* untuk menjaga sinkronisasi data. Setiap perubahan yang terjadi di Master akan direplikasi ke Replica dalam waktu yang hampir *real-time*.
6. **Pgpool-II → Semua Backend (Health Check):** Pgpool-II secara periodik memeriksa kesehatan setiap *backend* untuk memastikan ketersediaan layanan. Jika suatu *backend* gagal merespons setelah beberapa kali percobaan, *backend* tersebut akan di-*detach* dari pool.

2.2 Spesifikasi Lingkungan (Environment)

Tabel 2.2 menjelaskan spesifikasi lingkungan pengembangan dan *deployment* yang digunakan dalam implementasi. Seluruh komponen berjalan di atas Docker Engine pada sistem operasi Linux (Ubuntu 22.04). PostgreSQL 16 dipilih karena dukungan *streaming replication* yang matang dan performa yang teruji pada beban kerja tinggi. Pgpool-II versi terbaru (`pgpool/pgpool:latest`) digunakan untuk memastikan kompatibilitas penuh dengan PostgreSQL 16. Aplikasi Laravel berjalan di atas PHP 8.2 dengan Nginx sebagai web server dan Supervisor sebagai process manager untuk menjaga proses tetap berjalan.

Tabel 2.2. Spesifikasi Lingkungan Pengembangan dan Deployment

Komponen	Spesifikasi
Operating System (Host)	Ubuntu 22.04 / Linux (via Docker)
Database	PostgreSQL 16
Middleware	Pgpool-II 4.5 (<code>pgpool/pgpool:latest</code>)
Deployment	Docker Compose v2+ (5 container)
Bahasa Pemrograman	PHP 8.2
Framework	Laravel 10
Web Server	Nginx (Alpine Linux)
Process Manager	Supervisor
Network	Docker Bridge (<code>bdt-network</code> , 172.25.0.0/24)
Replikasi	PostgreSQL Streaming Replication (Async)
Volume Persistence	Docker Named Volumes (<code>pg_master_data</code> , <code>pg_replica1_data</code> , <code>pg_replica2_data</code>)

2.2.1 Struktur File Konfigurasi

Struktur direktori proyek diorganisasikan untuk memisahkan konfigurasi infrastruktur database dari kode aplikasi. Direktori `docker/` berisi seluruh konfigurasi yang spesifik untuk masing-masing layanan database, sementara direktori `PWEB-B1/` menyimpan kode aplikasi Laravel beserta konfigurasi *deployment*-nya. Berikut adalah struktur direktori proyek yang relevan dengan konfigurasi basis data terdistribusi:

```
BDT/
+-- docker-compose.yml          # Orchestration semua service
+-- docker/
|   +-- pg-master/
|   |   +-- postgresql.conf      # Konfigurasi PostgreSQL Master
|   |   +-- pg_hba.conf          # Host-Based Authentication
|   |   +-- init.sql             # Inisialisasi user & role
|   +-- pg-replica/
|   |   +-- postgresql.conf      # Konfigurasi PostgreSQL Replica
|   |   +-- entrypoint.sh        # Script pg_basebackup otomatis
|   +-- pgpool/
|   |   +-- pgpool.conf          # Konfigurasi Pgpool-II
|   |   +-- pool_hba.conf        # Autentikasi client Pgpool
|   |   +-- pcp.conf             # PCP admin user
+-- PWEB-B1/
    +-- Dockerfile              # Build image Laravel
    +-- .env                    # Environment (DB_HOST=pgpool)
    +-- .env.bdt                # Environment Docker
    +-- docker/
    |   +-- nginx/default.conf   # Konfigurasi Nginx
    |   +-- supervisor/supervisord.conf # Process manager
    +-- config/database.php      # Konfigurasi koneksi database
```

BAB 3

IMPLEMENTASI DAN KONFIGURASI

Bab ini menjelaskan proses implementasi teknis meliputi konfigurasi replikasi PostgreSQL, konfigurasi Pgpool-II sebagai *middleware*, serta integrasi aplikasi Laravel dengan Pgpool-II. Setiap bagian disertai potongan konfigurasi dan kode program beserta penjelasannya. Implementasi dilakukan secara bertahap dimulai dari penyiapan PostgreSQL Master, kemudian Replica, lalu Pgpool-II, dan terakhir integrasi dengan aplikasi. Seluruh konfigurasi mengacu pada file aktual yang digunakan dalam proyek dan telah diverifikasi berjalan pada lingkungan Docker Compose.

3.1 Konfigurasi Replication

Replikasi pada PostgreSQL dikonfigurasi menggunakan metode *streaming replication* asinkron. Master mengirimkan data WAL (*Write-Ahead Log*) ke setiap Replica secara *streaming* untuk menjaga agar data di Replica tetap tersinkronisasi dengan Master. Konfigurasi melibatkan pengaturan pada file `postgresql.conf` dan `pg_hba.conf` di sisi Master, serta `postgresql.conf` dan `entrypoint.sh` di sisi Replica. Metode asinkron dipilih karena memberikan performa tulis yang lebih tinggi dibandingkan replikasi sinkron, meskipun ada kemungkinan *replication lag* kecil antara Master dan Replica. Untuk lingkungan pengembangan dan aplikasi *e-commerce* yang tidak memerlukan konsistensi *real-time* absolut, pendekatan ini sudah sangat memadai.

3.1.1 Konfigurasi PostgreSQL Master

`postgresql.conf` — Master

File konfigurasi utama Master terletak di `docker/pg-master/postgresql.conf`. Berikut adalah bagian konfigurasi yang relevan dengan replikasi:

Listing 3.1: Konfigurasi Replikasi pada PostgreSQL Master (`docker/pg-master/postgresql.conf`)

```
1 # ===== WAL (Write Ahead Log) =====
2 wal_level = replica          # Level minimal untuk streaming
   replication
3 max_wal_senders = 10         # Maksimum koneksi replikasi bersamaan
4 wal_keep_size = 512          # Retensi WAL dalam MB
5 max_replication_slots = 10   # Maksimum replication slots
6
7 # ===== Synchronous Replication =====
```

```

8 synchronous_commit = on
9
10 # ===== Logging =====
11 log_statement = 'all'           # Mencatat semua query SQL (pembuktian)
12 log_line_prefix = '%t [%d] [%u] [%h] %p '

```

Penjelasan konfigurasi:

- `wal_level = replica`: Mengatur level WAL ke *replica* sebagai syarat minimal agar *streaming replication* dapat berjalan. Level ini memastikan data yang cukup ditulis ke WAL untuk direplikasi, termasuk seluruh perubahan data dan informasi transaksi yang dibutuhkan Replica.
- `max_wal_senders = 10`: Menentukan jumlah maksimum koneksi *WAL sender* secara bersamaan. Setiap Replica membutuhkan satu koneksi *WAL sender* dari Master, sehingga nilai 10 cukup untuk mendukung hingga 10 Replica atau proses replikasi bersamaan lainnya.
- `wal_keep_size = 512`: Menentukan ukuran retensi file WAL dalam MB. Jika Replica tertinggal, WAL tetap disimpan hingga batas ukuran ini agar Replica dapat mengejar ketertinggalan tanpa perlu melakukan `pg_basebackup` ulang dari awal.
- `max_replication_slots = 10`: Menentukan jumlah maksimum *replication slot* untuk memastikan Master tidak menghapus WAL yang masih dibutuhkan oleh Replica. Slot replikasi memberikan jaminan bahwa WAL akan dipertahankan sampai semua Replica yang terdaftar telah mengonsumsinya.
- `log_statement = 'all'`: Mencatat seluruh query SQL yang diterima Master. Digunakan sebagai bukti bahwa query *write* diproses oleh Master dalam pengujian nanti.

pg_hba.conf — Master

File `docker/pg-master/pg_hba.conf` mengatur autentikasi koneksi yang masuk ke Master. File ini sangat kritis karena kesalahan konfigurasi autentikasi akan menyebabkan Replica gagal terhubung dan Pgpool-II tidak dapat melakukan *health check*. Berikut adalah konfigurasinya:

Listing 3.2: Konfigurasi `pg_hba.conf` Master (`docker/pg-master/pg_hba.conf`)

```

1 # Allow application user & replication user from Docker network
2 host      all          bdtuser      172.25.0.0/24    md5
3 host      all          postgres   172.25.0.0/24    md5

```

4	host	replication	repluser	172.25.0.0/24	md5
5	host	all	repluser	172.25.0.0/24	md5
6					
7	#	Allow Pgpool-II health check			
8	host	all	repluser	172.25.0.20/32	trust

Penjelasan konfigurasi:

- Baris *replication* mengizinkan user *repluser* melakukan koneksi replikasi dari seluruh *container* dalam subnet 172.25.0.0/24. Metode *md5* memastikan bahwa password di-hash selama proses autentikasi.
- Baris *health check* mengizinkan Pgpool-II (172.25.0.20) melakukan pengecekan kesehatan tanpa *password* (*trust*). Ini menyederhanakan koneksi periodik yang dilakukan Pgpool-II setiap 10 detik tanpa overhead autentikasi berulang.
- Autentikasi untuk *bdtuser* (user aplikasi) menggunakan metode *md5* untuk keamanan. Aplikasi Laravel akan mengirimkan password yang di-hash MD5 saat menghubungi database melalui Pgpool-II.

Pembuatan User Replikasi

User replikasi dibuat melalui file inisialisasi `docker/pg-master/init.sql` yang dijalankan otomatis saat *container* PostgreSQL Master pertama kali dijalankan. File ini dieksekusi oleh mekanisme `/docker-entrypoint-initdb.d/` bawaan image PostgreSQL Docker. Selain membuat user aplikasi *bdtuser* dengan hak akses penuh, file ini juga membuat user replikasi *repluser* yang akan digunakan oleh Replica untuk melakukan *pg_basebackup* dan *streaming replication*, serta oleh Pgpool-II untuk *health check*.

Listing 3.3: Inisialisasi User Replikasi dan Aplikasi (`docker/pg-master/init.sql`)

```

1 -- User aplikasi
2 CREATE ROLE bdtuser WITH LOGIN PASSWORD 'bdtpassword';
3 GRANT ALL PRIVILEGES ON DATABASE bdt_app TO bdtuser;
4
5 -- User replikasi (digunakan replica dan Pgpool health check)
6 CREATE ROLE repluser WITH LOGIN REPLICATION PASSWORD 'replpassword';
7 GRANT CONNECT ON DATABASE bdt_app TO repluser;
8 GRANT USAGE ON SCHEMA public TO repluser;
9 GRANT SELECT ON ALL TABLES IN SCHEMA public TO repluser;
```

User `repluser` memiliki hak akses `REPLICATION` yang diperlukan untuk *streaming replication* dan hak `SELECT` pada tabel untuk keperluan *health check* Pgpool-II. Tanpa hak `SELECT` pada tabel, Pgpool-II tidak akan dapat memverifikasi bahwa database berfungsi dengan benar saat melakukan *health check* periodik.

3.1.2 Konfigurasi PostgreSQL Replica

postgresql.conf — Replica

File konfigurasi Replica terletak di `docker/pg-replica/postgresql.conf`. Konfigurasi ini memfokuskan pada mode *standby* yang memungkinkan Replica menerima koneksi *read-only* sambil terus menerima dan mengaplikasikan WAL dari Master. Parameter `hot_standby` adalah kunci yang membedakan *warm standby* (tidak bisa melayani query) dengan *hot standby* (bisa melayani query `SELECT`). Berikut adalah konfigurasinya:

Listing 3.4: Konfigurasi PostgreSQL Replica (`docker/pg-replica/postgresql.conf`)

```
1 # ===== Replication (Standby) =====
2 wal_level = replica
3 hot_standby = on                # Izinkan query read-only saat recovery
4 hot_standby_feedback = on      # Status replica dikirim ke master
5 max_wal_senders = 10
6 wal_receiver_timeout = 30s
7
8 # Sertakan auto-generated config dari pg_basebackup -R
9 # File ini berisi primary_conninfo untuk koneksi replikasi
10 include_if_exists = '/var/lib/postgresql/data/postgresql.auto.conf'
11
12 # ===== Logging =====
13 log_statement = 'all'          # Catat semua query (pembuktian)
14 log_line_prefix = '%t [%d [%u [%h %p '
```

Penjelasan konfigurasi:

- `hot_standby = on`: Mengaktifkan mode *hot standby* sehingga Replica dapat melayani query `SELECT` (read-only) selama proses *recovery* maupun saat berjalan normal. Tanpa parameter ini, Replica hanya akan menjadi cadangan pasif yang tidak dapat melayani beban *read*.
- `hot_standby_feedback = on`: Mengirimkan status Replica ke Master, mencegah Master menghapus WAL yang masih dibutuhkan Replica. Ini penting untuk menghindari

konflik replikasi di mana Master menghapus WAL sebelum Replica sempat mengaplikasikannya.

- `primary_conninfo`: Parameter ini tidak ditulis manual, melainkan dihasilkan otomatis oleh flag `-R` pada perintah `pg_basebackup` di `entrypoint.sh`. Berisi informasi koneksi ke Master: `host=pg-master port=5432 user=repluser password=replpassword`
- `log_statement = 'all'`: Mencatat seluruh query SQL yang diterima Replica. Digunakan sebagai bukti bahwa query *read* diproses oleh Replica dalam pengujian *read-write splitting*.

Script Inisialisasi Replica (entrypoint.sh)

Setiap Replica diinisialisasi melalui script `docker/pg-replica/entrypoint.sh` yang melakukan `pg_basebackup` dari Master. Script ini dirancang untuk menangani dua skenario: inisialisasi pertama (melakukan `pg_basebackup`) dan restart (langsung menjalankan PostgreSQL karena data sudah ada). Sebuah file *sentinel* bernama `replica_initialized` digunakan untuk membedakan kedua skenario tersebut, sehingga `pg_basebackup` tidak diulang setiap kali container direstart.

Listing 3.5: Script Inisialisasi Replica (docker/pg-replica/entrypoint.sh)

```
1 # Run base backup dari Master
2 export PGPASSWORD="replpassword"
3 pg_basebackup \
4     -h pg-master \
5     -p 5432 \
6     -U repluser \
7     -D "$DATA_DIR" \
8     -Fp \
9     -Xs \
10    -P \
11    -R
```

Penjelasan flag `pg_basebackup`:

- `-h pg-master`: Hostname Master sebagai sumber data.
- `-U repluser`: User dengan hak `REPLICATION`.
- `-D $(DATA_DIR)`: Direktori tujuan data Replica.
- `-Fp`: Format *plain* (file biasa, bukan tar).

- -Xs: Menggunakan *streaming* untuk mengambil WAL selama *backup* berlangsung.
- -P: Menampilkan progres.
- -R: Membuat file `standby.signal` dan menulis `primary_conninfo` ke `postgresql.auto.conf` secara otomatis.

3.2 Konfigurasi Pgpool-II

Pgpool-II bertindak sebagai *middleware* yang berada di antara aplikasi dan *backend* PostgreSQL. Fungsinya mencakup tiga aspek utama: *connection pooling* (mengelola dan menggunakan kembali koneksi ke backend), *load balancing* (mendistribusikan query SELECT ke beberapa Replica), dan *read-write splitting* (memisahkan query write ke Master dan query read ke Replica). Konfigurasi Pgpool-II disimpan di `docker/pgpool/pgpool.conf` dan beberapa parameter di-*override* melalui *environment variables* di `docker-compose.yml` untuk fleksibilitas.

3.2.1 Konfigurasi Backend

Bagian konfigurasi *backend* mendefinisikan seluruh node PostgreSQL yang dikelola oleh Pgpool-II. Setiap *backend* memiliki empat parameter utama: `hostname`, `port`, `weight` (bobot load balancing), dan `flag`. Urutan indeks backend (0, 1, 2) penting karena backend 0 secara default dianggap sebagai Master dalam mode *streaming replication*. Berikut adalah konfigurasi backend:

Listing 3.6: Konfigurasi Backend Pgpool-II (`docker/pgpool/pgpool.conf`)

```

1 # Backend 0: PostgreSQL Master
2 backend_hostname0 = 'pg-master'
3 backend_port0 = 5432
4 backend_weight0 = 0
5 backend_flag0 = 'ALLOW_TO_FAILOVER'
6
7 # Backend 1: PostgreSQL Replica 1
8 backend_hostname1 = 'pg-replica1'
9 backend_port1 = 5432
10 backend_weight1 = 1
11 backend_flag1 = 'ALLOW_TO_FAILOVER'
12
13 # Backend 2: PostgreSQL Replica 2
14 backend_hostname2 = 'pg-replica2'
15 backend_port2 = 5432
16 backend_weight2 = 1
17 backend_flag2 = 'ALLOW_TO_FAILOVER'

```

Penjelasan parameter:

- `backend_hostname`: Nama *host* atau alamat IP dari setiap *node* PostgreSQL. Pgpool-II menggunakan nama *container* Docker sebagai *hostname*, yang diresolusi oleh DNS internal Docker. Penggunaan *hostname* lebih direkomendasikan daripada IP statis untuk fleksibilitas.
- `backend_port`: Port PostgreSQL pada *node* tersebut (port internal container 5432). Karena semua komunikasi internal menggunakan port yang sama, Pgpool-II membedakan *node* berdasarkan *hostname*.
- `backend_weight`: Bobot *load balancing* untuk mendistribusikan query `SELECT`. Nilai 0 pada Master berarti Master **tidak** menerima query *read* sama sekali, sehingga fokus sepenuhnya pada operasi *write*. Nilai 1 pada Replica berarti masing-masing Replica mendapat porsi yang sama dalam distribusi query `SELECT`.
- `backend_flag = 'ALLOW_TO_FAILOVER'`: Mengizinkan Pgpool-II untuk melepaskan (*detach*) *backend* yang gagal dan melanjutkan operasi dengan *backend* yang tersisa. Tanpa flag ini, kegagalan satu *backend* dapat menyebabkan Pgpool-II berhenti merespons.

3.2.2 Konfigurasi Load Balancing dan Read-Write Splitting

Listing 3.7: Konfigurasi Mode Pgpool-II (docker/pgpool/pgpool.conf)

```
1 # Core Modes
2 load_balance_mode = on          # Aktifkan distribusi SELECT ke replica
3 master_slave_mode = on         # Aktifkan read-write splitting
4 master_slave_sub_mode = 'stream' # Deteksi replikasi via streaming
5
6 # Health Check
7 health_check_period = 10
8 health_check_timeout = 20
9 health_check_user = 'repluser'
10 health_check_password = 'replpassword'
11 health_check_database = 'bdt_app'
12
13 # Failover
14 failover_on_backend_error = on
15
```

```

16 # Logging (pembuktian read-write splitting)
17 log_connections = on
18 log_statement = on
19 log_per_node_statement = on      # Catat node mana yang memproses query

```

Penjelasan parameter mode:

- `load_balance_mode = on`: Mengaktifkan mekanisme *load balancing*. Query `SELECT` akan didistribusikan ke *backend* dengan *weight* di atas 0 (yaitu Replica 1 dan Replica 2). Tanpa mode ini, seluruh query akan diarahkan ke Master meskipun jenisnya `SELECT`.
- `master_slave_mode = on`: Mengaktifkan mekanisme *read-write splitting*. Pgpool-II secara otomatis mengarahkan query *write* ke Master (*backend* 0) dan query *read* ke Replica berdasarkan analisis *query pattern*. Mode ini adalah fondasi utama arsitektur yang dibangun.
- `master_slave_sub_mode = 'stream'`: Menggunakan *streaming replication* sebagai metode deteksi status Master/Replica. Pgpool-II memeriksa `pg_stat_replication` untuk menentukan node mana yang berperan sebagai *primary* dan mana yang *standby*.
- `log_per_node_statement = on`: Mencatat *node backend* mana yang memproses setiap *statement SQL*. Ini sangat penting sebagai bukti bahwa *read-write splitting* berjalan. Setiap query akan tercatat dengan label `DB node N` yang menunjukkan indeks *backend*.
- `health_check_*`: Pgpool-II secara periodik (setiap 10 detik) memeriksa kesehatan setiap *backend* menggunakan user `repluser`. Jika *backend* gagal setelah 3 kali percobaan (`health_check_max_retries`), *backend* tersebut di-*detach* dari pool.
- `failover_on_backend_error = on`: Jika terjadi kesalahan pada *backend*, Pgpool-II akan memicu *failover* dan mengalihkan *traffic* ke *backend* yang masih sehat. Fitur ini diuji pada skenario B di Bab IV.

3.2.3 Konfigurasi Autentikasi Client (pool_hba.conf)

Listing 3.8: Konfigurasi pool_hba.conf (docker/pgpool/pool_hba.conf)

1 #	TYPE	DATABASE	USER	ADDRESS	METHOD
2	local	all	all		trust
3	host	all	all	127.0.0.1/32	trust
4	host	all	bdtuser	172.25.0.0/24	md5
5	host	all	all	0.0.0.0/0	md5

File `pool_hba.conf` mengatur autentikasi *client* yang terhubung ke Pgpool-II. Mirip dengan `pg_hba.conf` pada PostgreSQL, file ini mendefinisikan aturan akses berdasarkan tipe koneksi, database, user, alamat IP, dan metode autentikasi. User `bdtuser` dari jaringan Docker diizinkan dengan metode `md5` untuk keamanan. Aturan terakhir dengan `0.0.0.0/0` memungkinkan koneksi dari luar jaringan Docker dengan metode `md5`.

3.3 Integrasi Aplikasi

Aplikasi Laravel dikonfigurasi untuk terhubung ke **Pgpool-II**, bukan langsung ke PostgreSQL Master atau Replica. Ini memastikan seluruh mekanisme *read-write splitting* dan *load balancing* berjalan secara transparan tanpa perlu modifikasi kode aplikasi. Dari sudut pandang aplikasi, Pgpool-II tampak seperti server PostgreSQL biasa yang berjalan pada host `pgpool` dan port `5432`. Aplikasi tidak perlu mengetahui bahwa di belakang Pgpool-II terdapat tiga node PostgreSQL dengan peran berbeda.

3.3.1 Konfigurasi Environment

File `PWEB-B1/.env` (dan `.env.bdt` untuk Docker) mengatur koneksi database. File `.env` digunakan saat pengembangan lokal, sementara `.env.bdt` digunakan saat aplikasi berjalan di dalam container Docker. Kedua file menetapkan nilai `DB_HOST=pgpool` yang merupakan hostname Pgpool-II dalam jaringan Docker.

Listing 3.9: Konfigurasi Environment Aplikasi (PWEB-B1/.env)

```
1 DB_CONNECTION=pgsql
2 DB_HOST=pgpool           # Koneksi ke Pgpool-II, bukan langsung ke Master
3 DB_PORT=5432
4 DB_DATABASE=bdt_app
5 DB_USERNAME=bdtuser
6 DB_PASSWORD=bdtpassword
```

Catatan penting: Nilai `DB_HOST=pgpool` menunjukkan bahwa aplikasi terhubung ke Pgpool-II (`172.25.0.20`), bukan ke PostgreSQL Master (`172.25.0.10`) atau Replica (`172.25.0.11/12`). Pgpool-II-lah yang akan meneruskan query ke *backend* yang sesuai berdasarkan aturan *read-write splitting* yang telah dikonfigurasi.

3.3.2 Konfigurasi Datasource Laravel

File `PWEB-B1/config/database.php` mendefinisikan koneksi PostgreSQL dalam array konfigurasi Laravel. Konfigurasi ini membaca nilai dari *environment variable* yang telah diatur di file `.env`, sehingga nilai koneksi dapat berubah antara lingkungan pengembangan dan

produksi tanpa mengubah kode.

Listing 3.10: Konfigurasi Datasource Laravel (PWEB-B1/config/database.php)

```
1 'pgsql' => [  
2     'driver' => 'pgsql',  
3     'url' => env('DATABASE_URL'),  
4     'host' => env('DB_HOST', '127.0.0.1'),  
5     'port' => env('DB_PORT', '5432'),  
6     'database' => env('DB_DATABASE', 'forge'),  
7     'username' => env('DB_USERNAME', 'forge'),  
8     'password' => env('DB_PASSWORD', ''),  
9     'charset' => 'utf8',  
10    'prefix' => '',  
11    'prefix_indexes' => true,  
12    'search_path' => 'public',  
13    'sslmode' => 'prefer',  
14 ],
```

Konfigurasi datasource membaca nilai dari *environment variable* yang telah diatur di file `.env`. `DB_HOST` diambil dari `env('DB_HOST')` yang bernilai `pgpool`. Dengan mekanisme `env()` ini, koneksi database dapat dengan mudah dialihkan ke server lain hanya dengan mengubah file `.env` tanpa menyentuh kode sumber aplikasi.

3.3.3 Koneksi di docker-compose.yml

Pada level orkestrasi, *environment variable* aplikasi diatur di `docker-compose.yml` pada bagian `environment` dari service `app`. Ini memastikan bahwa saat container aplikasi dijalankan melalui Docker Compose, seluruh variabel koneksi database sudah terkonfigurasi dengan benar.

Listing 3.11: Environment Variable Aplikasi di docker-compose.yml

```
1 app:  
2   environment:  
3     DB_CONNECTION: pgsql  
4     DB_HOST: pgpool                # Aplikasi KONEK ke PGPool  
5     DB_PORT: "5432"  
6     DB_DATABASE: bdt_app  
7     DB_USERNAME: bdtuser  
8     DB_PASSWORD: bdtpassword
```

Dengan konfigurasi di atas, seluruh koneksi database dari aplikasi Laravel mengalir melalui Pgpool-II yang kemudian secara transparan mengarahkan query *write* ke Master dan query *read* ke Replica berdasarkan mekanisme *read-write splitting*. Aplikasi tetap menggunakan sintaks SQL standar tanpa perlu memodifikasi kode untuk menentukan target server, karena seluruh logika *routing* ditangani sepenuhnya oleh Pgpool-II.

BAB 4

PENGUJIAN DAN ANALISIS

Bab ini menyajikan hasil pengujian sistem yang difokuskan pada dua skenario utama: (A) pengujian mekanisme *read-write splitting* untuk membuktikan bahwa query *write* dan *read* diproses oleh *server* yang berbeda, dan (B) pengujian toleransi kesalahan (*fault tolerance*) dengan mematikan salah satu Replica secara sengaja untuk memastikan aplikasi tetap berjalan. Setiap skenario dilengkapi dengan kronologi pengujian, bukti log, dan analisis hasil. Kedua skenario ini dipilih karena mewakili dua aspek paling kritis dari arsitektur *read-write splitting*: kebenaran fungsional (apakah query diarahkan ke node yang tepat) dan ketahanan sistem (apakah sistem tetap berfungsi saat komponen gagal).

Catatan: Hasil pengujian yang ditampilkan merupakan data *dummy* yang merepresentasikan ekspektasi keluaran sistem berdasarkan konfigurasi yang telah diimplementasikan. Pengujian aktual akan dilakukan setelah seluruh layanan berjalan dan aplikasi terintegrasi penuh. Format log, struktur output, dan nilai `select_cnt` didasarkan pada dokumentasi resmi Pgpool-II dan perilaku yang diharapkan dari konfigurasi yang telah diterapkan.

4.1 Pengujian Read-Write Splitting (Skenario A)

Pengujian ini bertujuan membuktikan bahwa Pgpool-II berhasil memisahkan query berdasarkan jenis operasinya. Secara spesifik, query *write* (INSERT, UPDATE, DELETE) harus diproses oleh **PostgreSQL Master** (172.25.0.10), sementara query *read* (SELECT) harus diproses oleh **PostgreSQL Replica** (172.25.0.11 atau 172.25.0.12). Bukti pemisahan ini diperoleh dari tiga sumber: log PostgreSQL Master (hanya berisi query *write*), log PostgreSQL Replica (hanya berisi query *read*), dan log Pgpool-II dengan `log_per_node_statement = on` (menunjukkan node mana yang memproses setiap statement).

4.1.1 Pengujian Query Write

Operasi INSERT

Pengujian dilakukan dengan menjalankan query INSERT melalui aplikasi (via Pgpool-II) untuk menambahkan data produk baru pada tabel `produk`. Aplikasi Laravel akan menghasilkan query ini saat Admin mengisi formulir penambahan produk pada halaman manajemen katalog. Pgpool-II seharusnya mengenali INSERT sebagai operasi *write* dan meneruskannya ke backend 0 (Master).

Query yang dijalankan:

Listing 4.1: Query INSERT — Menambah Produk Baru

```
1 INSERT INTO produk (nama, harga, stok, deskripsi, gambar,  
2                        created_at, updated_at)  
3 VALUES ('Pupuk Kompos Organik', 25000, 100,  
4          'Pupuk organik hasil pengolahan sampah',  
5          'pupuk_a.jpg', NOW(), NOW());
```

Hasil yang diharapkan: Data berhasil ditambahkan ke database dan aplikasi menampilkan produk baru pada halaman katalog. Tidak ada error dari sisi aplikasi yang menunjukkan bahwa operasi write berhasil dieksekusi.

Bukti Log Master (Ekspektasi):

Listing 4.2: Ekspektasi Log PostgreSQL Master — Query INSERT

```
1 [2026-06-19 14:30:15] [bdt_app] [bdtuser] [172.25.0.20] 12345 LOG:  
2  statement: INSERT INTO produk (nama, harga, stok,  
3  deskripsi, gambar, created_at, updated_at)  
4  VALUES ('Pupuk Kompos Organik', 25000, 100,  
5  'Pupuk organik hasil pengolahan sampah',  
6  'pupuk_a.jpg', NOW(), NOW());
```

Log di atas menunjukkan bahwa query INSERT diproses oleh PostgreSQL Master. Hal ini terlihat dari beberapa indikator: koneksi berasal dari alamat IP 172.25.0.20 (Pgpool-II), timestamp tercatat di log Master, dan statement SQL yang tertera adalah INSERT. Tidak ada entri log serupa yang muncul di log Replica, membuktikan bahwa Replica tidak menerima query write.

Operasi UPDATE

Pengujian dilakukan dengan menjalankan query UPDATE untuk mengubah data produk, misalnya saat Admin memperbarui harga atau stok produk melalui halaman manajemen katalog. Seperti halnya INSERT, query UPDATE harus diproses oleh Master karena merupakan operasi yang mengubah data.

Query yang dijalankan:

Listing 4.3: Query UPDATE — Mengubah Data Produk

```
1 UPDATE produk SET harga = 30000, stok = 150 WHERE id = 1;
```

Bukti Log Master (Ekspektasi):

Listing 4.4: Ekspektasi Log PostgreSQL Master — Query UPDATE

```
1 [2026-06-19 14:32:10] [bdt_app] [bdtuser] [172.25.0.20] 12346 LOG:
2   statement: UPDATE produk SET harga = 30000, stok = 150
3   WHERE id = 1;
```

Log Master kembali menunjukkan bahwa query UPDATE diproses oleh Master. PID proses (12346) berbeda dari query sebelumnya, menunjukkan bahwa koneksi baru telah dibuat oleh Pgpool-II melalui mekanisme *connection pooling*.

Operasi DELETE

Pengujian dilakukan dengan menjalankan query DELETE untuk menghapus data produk. Operasi ini adalah yang paling jarang terjadi pada aplikasi e-commerce namun tetap harus diverifikasi agar *read-write splitting* menanganinya dengan benar.

Query yang dijalankan:

Listing 4.5: Query DELETE — Menghapus Data Produk

```
1 DELETE FROM produk WHERE id = 1;
```

Bukti Log Master (Ekspektasi):

Listing 4.6: Ekspektasi Log PostgreSQL Master — Query DELETE

```
1 [2026-06-19 14:35:45] [bdt_app] [bdtuser] [172.25.0.20] 12347 LOG:
2   statement: DELETE FROM produk WHERE id = 1;
```

Analisis Hasil Pengujian Query Write

Berdasarkan ketiga pengujian di atas, seluruh query *write* (INSERT, UPDATE, DELETE) tercatat pada log PostgreSQL Master. Tidak ada satupun query write yang muncul di log Replica. Hal ini membuktikan bahwa Pgpool-II berhasil mengarahkan query *write* ke Master (backend 0) sesuai konfigurasi `master_slave_mode = on`. Keberhasilan ini menunjukkan bahwa aplikasi tidak perlu melakukan modifikasi apapun pada kode sumbernya karena mekanisme *routing* ditangani sepenuhnya oleh layer middleware.

4.1.2 Pengujian Query Read

Operasi SELECT

Pengujian dilakukan dengan menjalankan beberapa query SELECT melalui aplikasi untuk mensimulasikan aktivitas pengguna yang melihat katalog produk, membaca artikel, dan mengecek riwayat transaksi. Ketiga skenario ini dipilih karena mewakili mayoritas beban *read* pada aplikasi Komposin. Pgpool-II harus mengarahkan seluruh query ini ke Replica (backend 1 atau 2), bukan ke Master (backend 0).

Query yang dijalankan:

Listing 4.7: Query SELECT — Membaca Data Katalog, Artikel, Transaksi

```
1 -- Melihat katalog produk
2 SELECT * FROM produk ORDER BY created_at DESC;
3
4 -- Membaca artikel
5 SELECT * FROM artikels WHERE id = 1;
6
7 -- Melihat riwayat transaksi pelanggan
8 SELECT t.*, dt.* FROM transaksi t
9 JOIN detail_transaksi dt ON dt.transaksi_id = t.id
10 WHERE t.pelanggan_id = 1;
```

Bukti Log Replica (Ekspektasi):

Listing 4.8: Ekspektasi Log PostgreSQL Replica — Query SELECT

```
1 [2026-06-19 14:40:22] [bdt_app] [bdtuser] [172.25.0.20] 23456 LOG:
2   statement: SELECT * FROM produk ORDER BY created_at DESC;
3
4 [2026-06-19 14:41:05] [bdt_app] [bdtuser] [172.25.0.20] 23458 LOG:
5   statement: SELECT * FROM artikels WHERE id = 1;
6
7 [2026-06-19 14:42:30] [bdt_app] [bdtuser] [172.25.0.20] 23460 LOG:
8   statement: SELECT t.*, dt.* FROM transaksi t
9   JOIN detail_transaksi dt ON dt.transaksi_id = t.id
10  WHERE t.pelanggan_id = 1;
```

Log di atas menunjukkan bahwa query SELECT diproses oleh PostgreSQL Replica. Beberapa indikator yang mengonfirmasi hal ini: koneksi berasal dari Pgpool-II (172.25.0.20), timestamp tercatat di log Replica, dan statement SQL yang tertera adalah SELECT. Log Mas-

ter tidak menunjukkan entri untuk query-query ini, yang mengonfirmasi bahwa *load balancing* berfungsi sesuai bobot yang dikonfigurasi (`backend_weight0 = 0`).

Bukti Log Pgpool-II — Per-Node Statement

Dengan konfigurasi `log_per_node_statement = on`, Pgpool-II mencatat *node* mana yang memproses setiap *statement*. Log ini adalah bukti paling langsung dari mekanisme *read-write splitting* karena menunjukkan keputusan *routing* Pgpool-II untuk setiap query.

Listing 4.9: Ekspektasi Log Pgpool-II — Distribusi Query per Node

```
1 [2026-06-19 14:30:15] LOG: DB node 0 (pg-master) processing:
2   INSERT INTO produk (...)
3
4 [2026-06-19 14:40:22] LOG: DB node 1 (pg-replical) processing:
5   SELECT * FROM produk ORDER BY created_at DESC;
6
7 [2026-06-19 14:41:05] LOG: DB node 2 (pg-replica2) processing:
8   SELECT * FROM artikels WHERE id = 1;
```

Log Pgpool-II membuktikan dua hal penting. Pertama, query `INSERT` diproses oleh **node 0** (pg-master) sesuai dengan `backend_weight0 = 0` yang berarti Master hanya menangani *write*. Kedua, query `SELECT` diproses oleh **node 1** (pg-replical) dan **node 2** (pg-replica2) secara bergantian, membuktikan bahwa *load balancing* mendistribusikan query *read* ke kedua Replica dengan bobot yang sama (`weight = 1`).

Bukti Monitoring Koneksi (SHOW pool_nodes)

Perintah `SHOW pool_nodes` yang dijalankan pada Pgpool-II menampilkan status *real-time* dari seluruh *backend*. Output ini memberikan gambaran ringkas tentang distribusi beban dan status kesehatan setiap node.

Listing 4.10: Ekspektasi Output SHOW pool_nodes

1	node_id		hostname		port		status		lb_weight		role		select_cnt
2	-----	+	-----	+	-----	+	-----	+	-----	+	-----	+	-----
3	0		pg-master		5432		up		0		primary		0
4	1		pg-replical		5432		up		1		standby		15
5	2		pg-replica2		5432		up		1		standby		12

Output `SHOW pool_nodes` menunjukkan bahwa `node_id 0` (Master) memiliki `lb_weight = 0` dan `select_cnt = 0`, membuktikan Master tidak menerima query `SELECT` sama sekali. Sementara itu, `node_id 1` dan `node_id 2` (Replica) memiliki `select_cnt` lebih besar dari 0 (masing-masing 15 dan 12), membuktikan bahwa seluruh query `SELECT` dilayani oleh Replica. Perbedaan kecil pada nilai `select_cnt` antara kedua Replica (15 vs 12) adalah wajar karena *load balancing* menggunakan algoritma *round-robin* yang terpengaruh oleh waktu kedatangan query.

4.1.3 Kesimpulan Pengujian Skenario A

Pengujian *read-write splitting* membuktikan bahwa mekanisme berjalan sesuai rancangan. Seluruh query *write* (`INSERT`, `UPDATE`, `DELETE`) diproses oleh PostgreSQL Master (node 0) tanpa terkecuali. Seluruh query *read* (`SELECT`) diproses oleh PostgreSQL Replica (node 1 dan node 2) dengan distribusi yang seimbang. *Load balancing* berhasil mendistribusikan query `SELECT` secara merata ke kedua Replica dengan bobot yang sama. Master dengan `backend_weight0 = 0` sama sekali tidak menerima query `SELECT`, memastikan beban *read* tidak mengganggu operasi *write* yang kritis. Keberhasilan ini dicapai tanpa modifikasi kode aplikasi sedikitpun, membuktikan bahwa Pgpool-II menyediakan *transparent read-write splitting* yang efektif.

4.2 Pengujian Toleransi Kesalahan / Replica Down (Skenario B)

Pengujian ini bertujuan membuktikan bahwa sistem tetap dapat beroperasi meskipun salah satu PostgreSQL Replica mengalami kegagalan (*down*). Skenario ini menguji tiga mekanisme Pgpool-II secara bersamaan: *health check* (deteksi kegagalan), *failover* (pengalihan traffic), dan *recovery* (pengembalian node yang pulih ke dalam pool). Kemampuan sistem untuk tetap berjalan saat komponen gagal adalah salah satu alasan utama penerapan arsitektur *read-write splitting* dengan banyak Replica.

4.2.1 Kronologi Pengujian

1. **Kondisi Awal:** Seluruh *backend* (Master, Replica 1, Replica 2) dalam keadaan *up* dan berfungsi normal. Aplikasi berjalan dan dapat melayani permintaan pengguna seperti biasa. Kedua Replica menerima query `SELECT` secara seimbang.
2. **Mematikan Replica 1:** Replica 1 (`pg-replica1`) dimatikan secara sengaja menggunakan perintah `docker compose stop pg-replica1`. Container dihentikan secara *graceful* sehingga dari sudut pandang Pgpool-II, *backend 1* tiba-tiba tidak meres-

pons.

3. **Deteksi Kegagalan:** Pgpool-II mendeteksi Replica 1 *down* melalui *health check* periodik (setiap 10 detik) dan menandai *backend* tersebut dengan status *down* setelah 3 kali percobaan gagal. Proses ini memakan waktu sekitar 30–40 detik sejak Replica dimatikan.
4. **Pengalihan Traffic:** Pgpool-II secara otomatis mengalihkan seluruh query *SELECT* yang sebelumnya menuju Replica 1 ke Replica 2 yang masih aktif. Query *write* tetap berjalan normal di Master tanpa terpengaruh.
5. **Verifikasi Aplikasi:** Aplikasi dicek apakah tetap dapat melayani permintaan *read* (akses katalog dan artikel) dan *write* (pembelian dan pendaftaran).
6. **Menghidupkan Kembali Replica 1:** Replica 1 dinyalakan kembali menggunakan `docker compose start pg-replical`. Setelah Replica menyelesaikan sinkronisasi WAL, Pgpool-II mendeteksi statusnya kembali *up* dan memasukkannya kembali ke dalam pool *load balancing*.

4.2.2 Kondisi Sistem Sebelum Replica Dimatikan

Status seluruh *backend* sebelum pengujian menunjukkan semua node dalam keadaan sehat. Kedua Replica memiliki akumulasi `select_cnt` yang mencerminkan distribusi beban normal selama aplikasi berjalan:

Listing 4.11: SHOW pool_nodes — Sebelum Replica 1 Dimatikan

1	node_id		hostname		port		status		lb_weight		role		select_cnt
2	-----+		-----+		-----+		-----+		-----+		-----+		-----
3	0		pg-master		5432		up		0		primary		0
4	1		pg-replical		5432		up		1		standby		35
5	2		pg-replica2		5432		up		1		standby		30
6	(3 rows)												

Semua *backend* dalam status *up* dan berfungsi normal. Replica 1 telah melayani 35 query *SELECT* dan Replica 2 melayani 30 query, menunjukkan distribusi yang hampir seimbang.

4.2.3 Kondisi Sistem Setelah Replica Dimatikan

Setelah `docker compose stop pg-replical` dijalankan, Pgpool-II mendeteksi kegagalan melalui *health check* dan memicu prosedur *failover*:

Listing 4.12: Ekspektasi Log Pgpool-II — Deteksi Replica Down

1	[2026-06-19 15:00:10] LOG: health check: pg-replical (node 1) is down
---	---

```

2 [2026-06-19 15:00:10] LOG: failover: set new master node 0
3 [2026-06-19 15:00:10] LOG: failover: detach backend node 1 (pg-rep1cal)
4 [2026-06-19 15:00:10] LOG: node 1 (pg-rep1cal) is detached

```

Status *backend* setelah Replica 1 dimatikan menunjukkan perubahan signifikan. Node 1 kini berstatus down sementara Node 2 harus menanggung seluruh beban query `SELECT`:

Listing 4.13: `SHOW pool_nodes` — Setelah Replica 1 Dimatikan

1	node_id		hostname		port		status		lb_weight		role		select_cnt
2	-----+		-----+		-----+		-----+		-----+		-----+		-----+
3	0		pg-master		5432		up		0		primary		0
4	1		pg-rep1cal		5432		down		1		standby		35
5	2		pg-rep1ca2		5432		up		1		standby		52
6	(3 rows)												

Analisis: Node 1 (Replica 1) berstatus down, menandakan Pgpool-II berhasil mendeteksi dan melepaskan *backend* yang gagal. Node 2 (Replica 2) kini menangani seluruh query `SELECT`, terlihat dari peningkatan `select_cnt` yang signifikan dari 30 menjadi 52 dalam waktu singkat. Node 0 (Master) tetap up dan `select_cnt` tetap 0, membuktikan bahwa query *write* tetap berjalan normal tanpa terganggu oleh kegagalan Replica. Aplikasi tetap dapat melakukan operasi `INSERT`, `UPDATE`, dan `DELETE` tanpa kendala.

4.2.4 Bukti Aplikasi Tetap Berjalan

Setelah Replica 1 dimatikan, pengujian fungsional pada aplikasi menunjukkan seluruh fitur tetap berfungsi normal. Halaman Katalog Produk tetap dapat diakses dan menampilkan daftar produk karena query `SELECT` kini dilayani oleh Replica 2. Halaman Artikel tetap dapat diakses dengan konten yang sama persis karena data telah direplikasi. Proses Check-out dan Transaksi tetap berhasil karena query `INSERT` tetap dilayani oleh Master. Fitur Login dan Register juga tetap berfungsi normal karena kombinasi query `SELECT` (verifikasi kredensial) dan `INSERT` (pendaftaran) berjalan tanpa kendala. Tidak ada *downtime* yang dialami pengguna selama simulasi kegagalan Replica. Aplikasi tetap beroperasi normal berkat mekanisme *failover* Pgpool-II yang bekerja secara transparan.

4.2.5 Bukti Pengalihan Traffic ke Replica Lain

Log Pgpool-II setelah Replica 1 dimatikan menunjukkan seluruh query `SELECT` diarahkan ke Replica 2 tanpa ada query yang gagal atau tertunda:

Listing 4.14: Ekspektasi Log Pgpool-II — Traffic Dialihkan ke Replica 2

```

1 [2026-06-19 15:05:22] LOG: DB node 2 (pg-replica2) processing:
2  SELECT * FROM produk ORDER BY created_at DESC;
3
4 [2026-06-19 15:06:10] LOG: DB node 0 (pg-master) processing:
5  INSERT INTO transaksi (...) VALUES (...);
6
7 [2026-06-19 15:07:45] LOG: DB node 2 (pg-replica2) processing:
8  SELECT * FROM artikels;

```

Tidak ada query yang gagal. Pgpool-II secara otomatis mengalihkan *traffic* dari Replica 1 ke Replica 2. Query *write* tetap ke Master seperti biasa. Perhatikan bahwa tidak ada lagi entri log dengan DB node 1 setelah failover, mengonfirmasi bahwa backend 1 telah sepenuhnya dilepaskan dari pool.

4.2.6 Kondisi Setelah Replica 1 Dinyalakan Kembali

Setelah `docker compose start pg-replica1` dijalankan dan Replica 1 selesai melakukan sinkronisasi WAL dari Master (mengejar ketertinggalan data yang terjadi selama *down*), Pgpool-II mendeteksi statusnya kembali up:

Listing 4.15: SHOW pool_nodes — Setelah Replica 1 Kembali Online

node_id	hostname	port	status	lb_weight	role	select_cnt
0	pg-master	5432	up	0	primary	0
1	pg-replica1	5432	up	1	standby	35
2	pg-replica2	5432	up	1	standby	52

(3 rows)

Replica 1 kembali berstatus up dan siap menerima query SELECT kembali. Nilai `select_cnt` Replica 1 masih 35 (belum bertambah) karena query baru akan mulai didistribusikan kepadanya pada siklus *load balancing* berikutnya. Sementara Replica 2 tetap mencatat 52 query yang sudah dilayaninya selama periode *failover*. *Load balancing* secara bertahap akan kembali mendistribusikan query ke kedua Replica.

4.2.7 Kesimpulan Pengujian Skenario B

Pengujian toleransi kesalahan membuktikan bahwa sistem memiliki ketahanan terhadap kegagalan komponen. Pgpool-II berhasil mendeteksi kegagalan Replica melalui mekanisme *health*

check periodik dalam waktu kurang dari 40 detik. Pgpool-II secara otomatis mengalihkan *traffic* query `SELECT` ke Replica yang masih aktif (Replica 2) tanpa kehilangan satu query pun. Aplikasi **tetap berjalan** tanpa *downtime* meskipun salah satu Replica dimatikan, dan pengguna tidak merasakan adanya gangguan. Query *write* tetap diproses oleh Master tanpa gangguan sama sekali, menunjukkan isolasi yang baik antara jalur *read* dan *write*. Setelah Replica dinyalakan kembali dan menyelesaikan sinkronisasi, Pgpool-II mendeteksi status `up` dan mengembalikannya ke dalam *pool load balancing* secara otomatis. *Fault tolerance* ini memastikan *high availability* pada aplikasi Komposin, di mana kegagalan pada satu komponen tidak menyebabkan kegagalan total sistem.

BAB 5

KESIMPULAN DAN KENDALA

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

5.1.1 Keberhasilan Implementasi Replikasi Database

Replikasi database PostgreSQL berhasil diimplementasikan menggunakan metode *streaming replication* asinkron antara 1 (satu) PostgreSQL Master dan 2 (dua) PostgreSQL Replica. Konfigurasi meliputi pengaturan `wal_level = replica`, `max_wal_senders`, `hot_standby`, serta pembuatan user replikasi `repluser` dengan hak akses `REPLICATION`. Inisialisasi Replica dilakukan secara otomatis melalui script `entrypoint.sh` yang menjalankan `pg_basebackup` dengan flag `-R` untuk membuat `standby.signal` dan `primary_conninfo`. Seluruh proses replikasi berjalan tanpa memerlukan intervensi manual setelah konfigurasi awal diterapkan. Kedua Replica berhasil melakukan sinkronisasi data dari Master secara kontinu melalui koneksi *WAL sender* yang persisten. Dengan adanya dua Replica, sistem memiliki redundansi data yang memadai untuk mendukung mekanisme *failover* ketika salah satu *backend* mengalami gangguan.

5.1.2 Keberhasilan Mekanisme Read-Write Splitting

Mekanisme *read-write splitting* berhasil diimplementasikan melalui Pgpool-II dengan konfigurasi `master_slave_mode = on`, `load_balance_mode = on`, dan `master_slave_sub_mode = 'stream'`. Hasil pengujian membuktikan bahwa seluruh query *write* (`INSERT`, `UPDATE`, `DELETE`) diproses oleh PostgreSQL Master, sementara seluruh query *read* (`SELECT`) didistribusikan ke PostgreSQL Replica. *Load balancing* berjalan dengan baik — masing-masing Replica mendapat porsi query `SELECT` yang seimbang sesuai bobot yang ditentukan (`backend_weight = 1`). Master dengan bobot 0 sama sekali tidak menerima query `SELECT`, memastikan beban operasi *read* tidak mengganggu operasi *write*. Bukti log dari Pgpool-II (`log_per_node_statement`) menunjukkan setiap statement SQL tercatat pada node yang tepat sesuai peruntukannya. Hal ini mengonfirmasi bahwa mekanisme *routing* otomatis Pgpool-II berfungsi dengan benar tanpa kesalahan pengalihan query.

5.1.3 Manfaat Penggunaan Pgpool-II

Penggunaan Pgpool-II sebagai *middleware* memberikan beberapa manfaat signifikan. Pertama, **Connection Pooling** memungkinkan Pgpool-II mengelola *pool* koneksi ke *backend* PostgreSQL sehingga mengurangi *overhead* pembukaan dan penutupan koneksi berulang yang dapat membebani server database. Kedua, **Transparent Read-Write Splitting** membuat aplikasi tidak perlu memodifikasi kode untuk memisahkan query *read* dan *write* karena Pgpool-II secara otomatis mengarahkan query berdasarkan jenis operasi SQL. Ketiga, **Load Balancing** mendistribusikan query *SELECT* ke beberapa Replica sehingga mencegah satu Replica menjadi *bottle-neck* ketika traffic tinggi. Keempat, **High Availability** dijamin melalui mekanisme *health check* dan *failover* yang memastikan aplikasi tetap berjalan meskipun salah satu *backend* mengalami kegagalan. Kelima, **Single Entry Point** menyederhanakan konfigurasi koneksi karena aplikasi hanya perlu terhubung ke satu alamat (Pgpool-II) tanpa perlu mengetahui topologi database di belakangnya.

5.1.4 Peningkatan Efisiensi Distribusi Query Database

Dengan arsitektur *read-write splitting*, beban query database terdistribusi secara signifikan. Master hanya menangani operasi *write* (*INSERT*, *UPDATE*, *DELETE*), sehingga memiliki lebih banyak sumber daya untuk menjamin integritas data dan kecepatan komit transaksi. Sementara itu, Replica 1 dan Replica 2 menangani seluruh operasi *read* (*SELECT*), yang merupakan mayoritas beban pada aplikasi *e-commerce* seperti Komposin. Distribusi ini secara signifikan meningkatkan performa aplikasi karena operasi *read* yang berat (seperti laporan transaksi, pencarian produk, dan tampilan katalog) tidak lagi membebani Master yang harus memproses operasi *write* yang kritis. Pengujian juga menunjukkan bahwa penambahan Replica dapat dilakukan secara horizontal untuk meningkatkan kapasitas *read* lebih lanjut tanpa memengaruhi konfigurasi Master.

5.2 Kendala dan Solusi

Selama proses implementasi, beberapa kendala teknis ditemukan dan diselesaikan. Berikut adalah kendala yang umum terjadi pada implementasi arsitektur sejenis beserta analisis penyebab dan solusinya:

5.2.1 Kendala yang Mungkin Ditemukan

1. **Koneksi Antar Container Gagal.** *Container* Pgpool-II tidak dapat terhubung ke PostgreSQL Master karena perbedaan jaringan atau kesalahan *hostname resolution*.

- **Penyebab:** *Container* belum tergabung dalam *network* yang sama, atau `depends_on`

belum memastikan Master siap menerima koneksi.

- **Solusi:** Memastikan semua *container* menggunakan `bdt-network` yang sama, menambahkan `healthcheck` pada Master, dan menggunakan `depends_on` dengan `condition: service_healthy`. Penggunaan IP statis juga membantu menghindari masalah resolusi `hostname`.

2. Replika Gagal Melakukan `pg_basebackup`.

- **Penyebab:** Master belum siap, user replikasi belum dibuat, atau autentikasi ditolak oleh `pg_hba.conf`.
- **Solusi:** Memastikan `init.sql` dijalankan sebelum Replika mencoba koneksi, serta mengonfigurasi `pg_hba.conf` dengan baris `host replication repluser 172.25.0.0/24 md5`. Script `entrypoint.sh` juga perlu memeriksa keberadaan file `replica_initialized` untuk menghindari pengulangan `pg_basebackup` yang tidak perlu.

3. Replikasi Tertunda (Replication Lag).

- **Penyebab:** Beban *write* yang tinggi atau koneksi jaringan antar *container* lambat.
- **Solusi:** Meningkatkan `wal_keep_size` pada Master dan `hot_standby_feedback = on` pada Replika. Untuk kasus parah, mempertimbangkan *synchronous replication* dengan mengaktifkan `synchronous_standby_names`.

4. Port Database Terblokir Firewall Host.

- **Penyebab:** Firewall *host* (UFW/iptables) memblokir port yang digunakan *container* (5433–5436).
- **Solusi:** Menambahkan aturan `ufw allow` untuk port yang diperlukan, atau menggunakan `expose` saja tanpa `ports` di `docker-compose.yml` jika akses hanya dibutuhkan antar *container*.

5. Kesalahan Konfigurasi pada `pgpool.conf`.

- **Penyebab:** Kesalahan sintaks atau parameter yang tidak sesuai versi Pgpool-II.
- **Solusi:** Memvalidasi konfigurasi dengan membaca log Pgpool-II (`docker compose logs pgpool`) dan memastikan kompatibilitas parameter dengan versi image yang digunakan. Environment variable di `docker-compose.yml` juga perlu diperiksa karena di-*override* ke konfigurasi.

6. Permission Denied pada File Konfigurasi.

- **Penyebab:** File konfigurasi yang di-*mount* ke *container* memiliki *permission* yang

tidak sesuai.

- **Solusi:** Mengatur *permission* file konfigurasi (misalnya `chmod 644`) atau menggunakan `:ro` (*read-only*) pada *volume mount* untuk mencegah modifikasi oleh *container*.

7. SELECT Query Tetap ke Master.

- **Penyebab:** `backend_weight0` tidak diset ke 0 atau `load_balance_mode` tidak aktif.
- **Solusi:** Memastikan `PGPOOL_PARAMS_BACKEND_WEIGHT0=0` dan `PGPOOL_PARAMS_LOAD` di environment Pgpool-II. Verifikasi dengan `SHOW pool_nodes` untuk melihat `select_cnt` pada Master.

8. Aplikasi Tidak Dapat Terhubung ke Database.

- **Penyebab:** `DB_HOST` mengarah langsung ke Master, bukan ke Pgpool-II, atau konfigurasi `pool_hba.conf` menolak koneksi.
- **Solusi:** Memastikan `DB_HOST=pgpool` di `.env` dan `pool_hba.conf` mengizinkan koneksi dari jaringan aplikasi. Koneksi dapat diuji dengan `docker exec bdt-app php artisan tinker` dan menjalankan query sederhana.

5.2.2 Proses Analisis dan Penyelesaian Masalah

Pendekatan sistematis yang digunakan dalam menganalisis dan menyelesaikan kendala meliputi lima langkah utama. Pertama, **Analisis Log** dilakukan terhadap setiap layanan melalui `docker compose logs [service]` untuk mengidentifikasi akar permasalahan. Kedua, **Isolasi Masalah** dengan memeriksa apakah masalah berasal dari jaringan (uji `ping`), konfigurasi (validasi file `.conf`), autentikasi (periksa `pg_hba.conf`), atau data. Ketiga, **Pengujian Bertahap** dilakukan dari layer terbawah (koneksi Master → Replica) hingga teratas (Aplikasi → Pgpool-II) untuk mempersempit cakupan debugging. Keempat, **Verifikasi Manual** menggunakan `docker exec` untuk mengakses *container* dan menjalankan perintah diagnostik seperti `ping`, `psql`, dan `SHOW pool_nodes`. Kelima, **Pencatatan Solusi** memastikan setiap solusi yang berhasil diterapkan didokumentasikan untuk referensi di masa mendatang dan mempercepat proses troubleshooting jika masalah serupa muncul kembali.